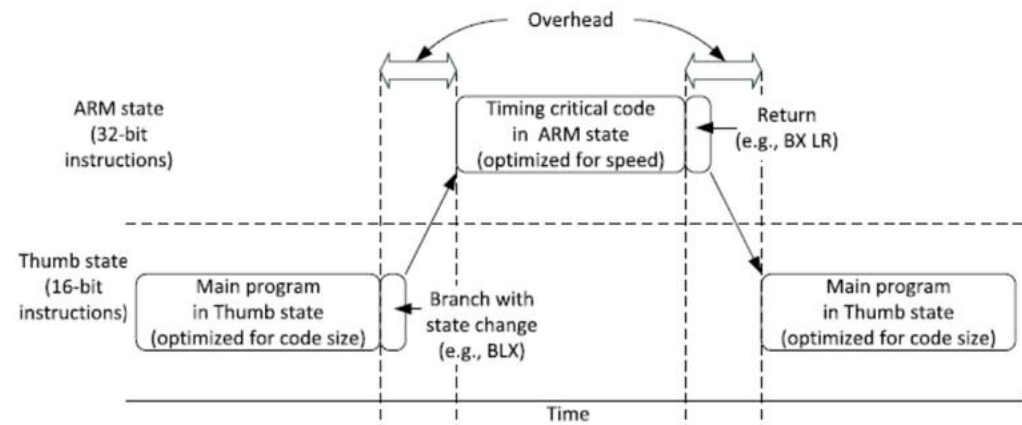


- Classic ARM processors were supporting both ARM 32 bit Instructions and Thum 16 bit instructions

**FIGURE 3.1**

Switching between ARM code and Thumb code in class ARM processors such as the ARM7TDMI

- **Thumb is a 16-bit instruction set**

- Optimised for code density from C code (~65% of ARM code size)
- Improved performance from narrow memory
- Subset of the functionality of the ARM instruction set

- **Core has additional execution state - Thumb**

- Switch between ARM and Thumb using **BX** instruction

ADDS r2,r2,#1

32-bit ARM Instruction

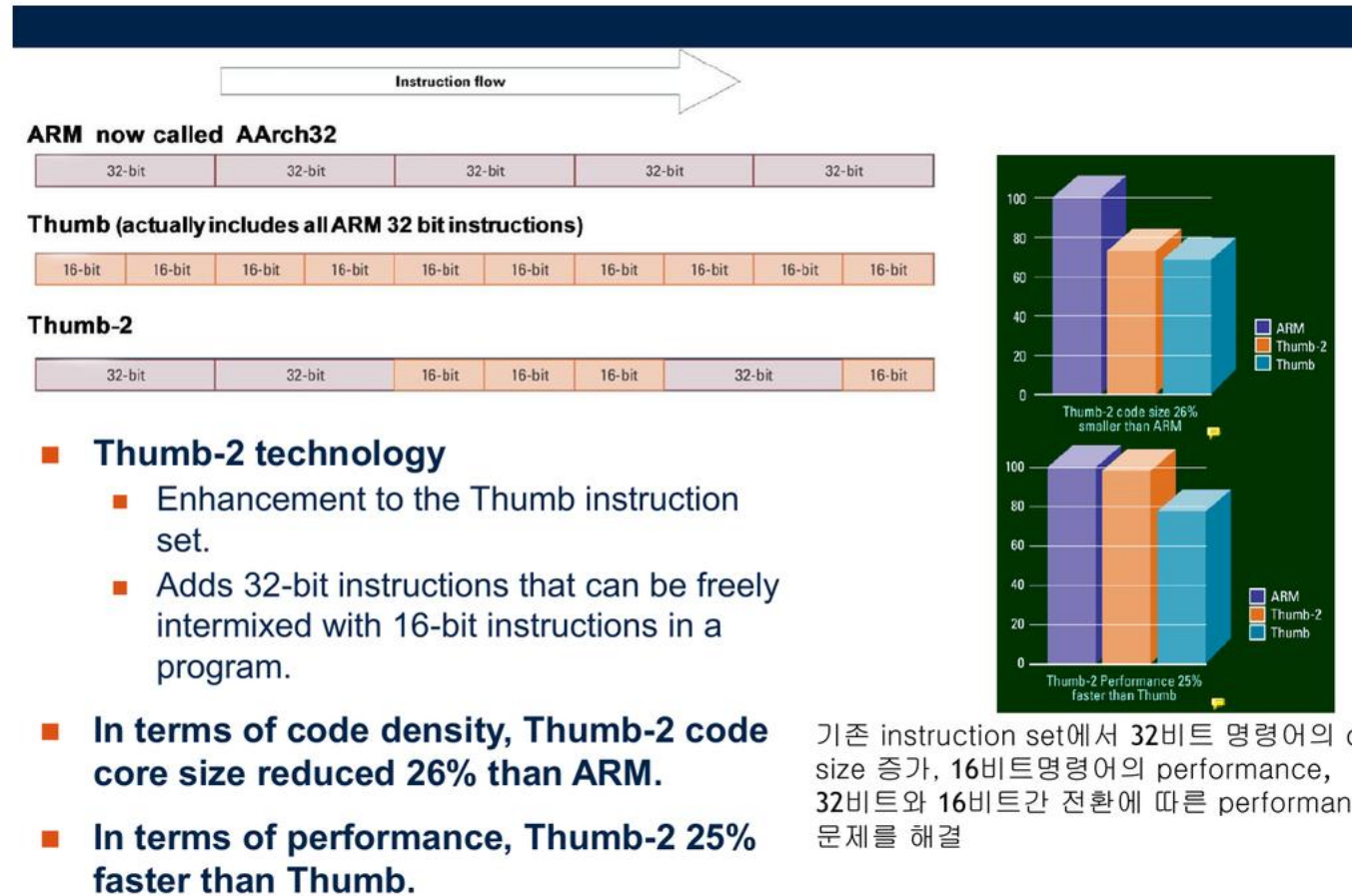


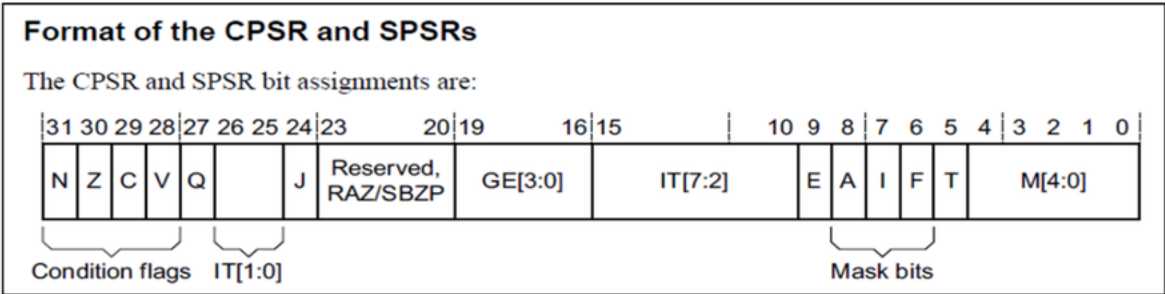
ADD r2,#1

16-bit Thumb Instruction

For most instructions generated by compiler:

- Conditional execution is not used
- Source and destination registers identical
- Only Low registers used
- Constants are of limited size
- Inline barrel shifter not used





```
if (r0 == 0)                cmp r0, #0
    r1 = r1 + 1;             ite EQ
else                          addeq r1, #1
```

Table A2-2 Effect of IT execution state bits

IT bits ^a						Note
[7:5]	[4]	[3]	[2]	[1]	[0]	
cond_base	P1	P2	P3	P4	1	Entry point for 4-instruction IT block
cond_base	P1	P2	P3	1	0	Entry point for 3-instruction IT block
cond_base	P1	P2	1	0	0	Entry point for 2-instruction IT block
cond_base	P1	1	0	0	0	Entry point for 1-instruction IT block
000	0	0	0	0	0	Normal execution, not in an IT block

- 예시)
- IT 필드는 **Thumb-2** 명령어인 **If-Then** 블록의 **Condition**과 명령어의 순서 번호를 가짐
 - CMP R0, R1
 - ITTEE EQ
 - ADDEQ R2, R0, R1 ; R2 = R0 + R1
 - ADDEQ R2, R0, R3 ; R2 = R0 + R3
 - SUBNE R2, R0, R1 ; R2 = R0 - R1
 - SUBNE R2, R0, R3 ; R2 = R0 - R3
- **R0 - R1** 한 결과가 "**0**" 이면 **ADDEQ** 명령어 2개를 실행하고 "**0**" 이 아니면 **SUBNE** 명령어 2개를 실행 하라는 의미

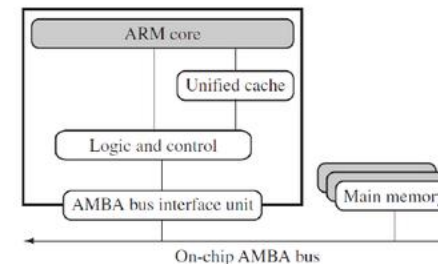
```
if( R0 == R1 )
{
    R2 = R0 + R1
    R2 = R0 + R3
}
else
{
    R2 = R0 - R1
    R2 = R0 - R3
}
```

2.5.1 CACHE AND TIGHTLY COUPLED MEMORY

- **Cache and Tightly Coupled Memory (TCM)**
- **The cache is a block of fast memory placed between main memory and the core. It allows for more efficient fetches from some memory types. With a cache the processor core can run for the majority of the time without having to wait for data from slow external memory.**
- **Most ARM-based embedded systems use a single-level cache internal to the processor.**

The cache is a block of fast memory placed between main memory and the core. It allows for more efficient fetches from some memory types. With a cache the processor core can run for the majority of the time without having to wait for data from slow external memory. Most ARM-based embedded systems use a single-level cache internal to the processor. Of course, many small embedded systems do not require the performance gains that a cache brings.

ARM has two forms of cache. The first is found attached to the Von Neumann-style cores. It combines both data and instruction into a single unified cache, as shown in Figure 2.13. For simplicity, we have called the glue logic that connects the memory system to the AMBA bus *logic and control*.



p.34, A simplified Von Neumann architecture with cache

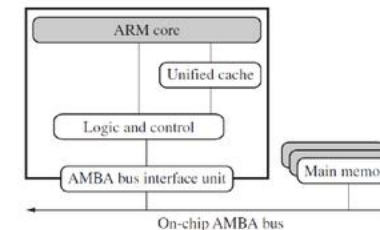
■ Cache

- **The cache is a block of fast memory placed between main memory and the core. It allows for more efficient fetches from some memory types. With a cache the processor core can run for the majority of the time without having to wait for data from slow external memory.**
- **Most ARM-based embedded systems use a single-level cache internal to the processor.**
- **ARM has two forms of cache.**
 - The first is found attached to the Von Neumann-style cores. It combines both data and instruction into a single unified cache, as shown in Figure.

CACHE AND TIGHTLY COUPLED MEMORY

The cache is a block of fast memory placed between main memory and the core. It allows more efficient fetches from some memory types. With a cache the processor core can run for the majority of the time without having to wait for data from slow external memory. Most ARM-based embedded systems use a single-level cache internal to the processor. Of course, many small embedded systems do not require the performance gains that a cache brings.

ARM has two forms of cache. The first is found attached to the Von Neumann-style cores. It combines both data and instruction into a single unified cache, as shown in Figure 2.13. For simplicity, we have called the glue logic that connects the memory system to the AMBA bus *logic and control*.



p.34, A simplified Von Neumann architecture with cache

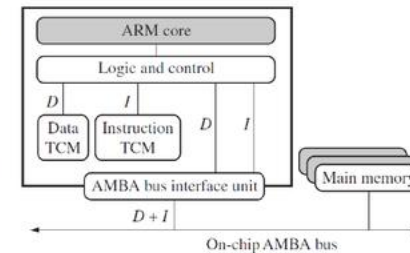
■ Tightly Coupled Memory (TCM)

- A cache provides an overall increase in performance but at the expense of predictable execution.
- [The second form, attached to the Harvard-style cores](#), has separate caches for data and instruction.
- Code execution is deterministic—the time taken for loading and storing instructions or data must be predictable.
- This is achieved using a form of memory called tightly coupled memory (TCM).

TCM is fast SRAM located close to the core and guarantees the clock cycles required to fetch instructions or data—critical for real-time algorithms requiring deterministic behavior.

The instruction and data TCMs have been integrated into the core. There are separate paths to each TCM, one for fetching instructions and one for

p.35, A simplified Harvard architecture with TCMs.



A simplified Harvard architecture with TCMs.

By contrast, the second form, attached to the Harvard-style cores, has separate caches for data and instruction.

A cache provides an overall increase in performance but at the expense of predictable execution. But for real-time systems it is paramount that code execution is *deterministic*—the time taken for loading and storing instructions or data must be predictable. This is achieved using a form of memory called *tightly coupled memory* (TCM). TCM is fast SRAM located close to the core and guarantees the clock cycles required to fetch instructions or data—critical for real-time algorithms requiring deterministic behavior. TCMs appear in the address map and can be accessed as fast memory. An example of a process with TCMs is shown in Figure 2.14.

■ MPUs

- employ a simple system that uses a limited number of memory regions.
- These regions are controlled with a set of special coprocessor registers, and each region is defined with specific access permissions.
- This type of memory management is used for systems that require memory protection but don't have a complex memory map.
- The MPU is explained in Chapter 13.

■ MMUs

- The most comprehensive memory management hardware available on the ARM.
- Uses a set of translation tables to provide fine-grained control over memory. These tables are stored in main memory and provide a virtual-to-physical address map as well as access permissions.
- The MMU is explained in Chapter 14.

■ Coprocessors

- can be attached to the ARM processor.
- extends the processing features of a core by extending the instruction set or by providing configuration registers.
- More than one coprocessor can be added to the ARM core via the coprocessor interface.

■ 코어의 처리기능을 확장

- 명령어 집합을 확장하거나
- 설정 레지스터를 제공

■ 벡터 부동소수점 연산(Vector Floating-Point :VFP)

- IEEE754.1985 부동소수점 표준과 호환
- CP10/CP11

■ Coprocessors

- can be accessed through a group of dedicated ARM instructions that provide a load-store type interface. Consider, for example, coprocessor 15: The ARM processor uses coprocessor 15 registers to control the cache, TCMs, and memory management.
- The coprocessor can also extend the instruction set by providing a specialized group of new instructions. For example, there are a set of specialized instructions that can employ,

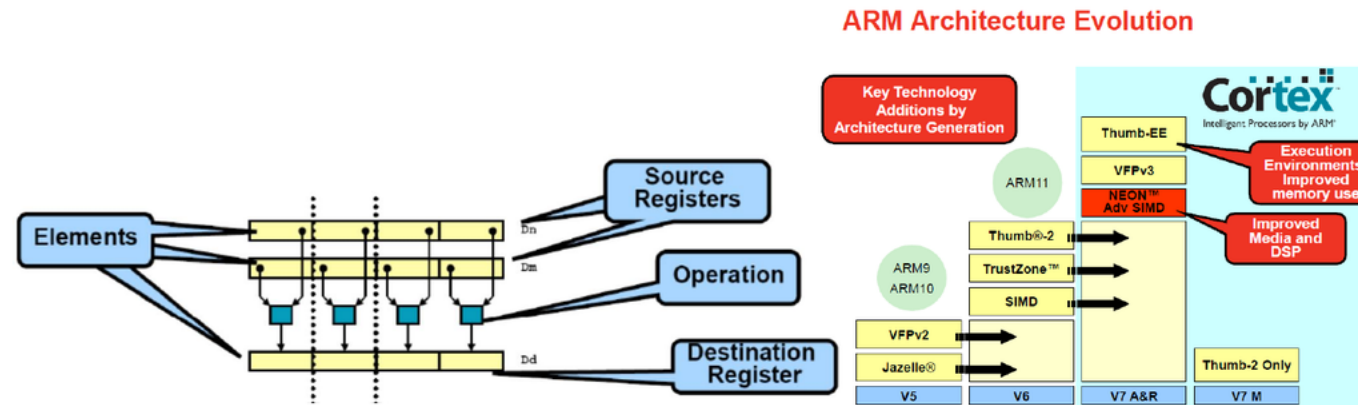
- 시스템 제어 코프로세서(**System Control Coprocessor**)
 - CP15
 - Controls the cache, TCMs, MPU, MMU
 - mcr : Move from CPU Register to Co-processor register
 - ARM 레지스터 정보를 Co-processor 레지스터로 가져옴
 - mrc : Move from Co-processor register to CPU Register
 - Co-processor 레지스터 정보를 ARM 레지스터로 가져옴
 - msr : CPSR에 있는 내용을 세팅한다.

```
MCR{cond} P15, <Opcode_1>, <Rd>, <CRn>, <CRm>, <Opcode_2>  
MRC{cond} P15, <Opcode_1>, <Rd>, <CRn>, <CRm>, <Opcode_2>
```

■ VFP and NEON(VFP v3)

- NEON is a wide SIMD data processing architecture.
- Extension of ARM instruction sets, 32 registers, 64-bit wide
- Registers are considered as vectors of elements of the same data type
- Instructions perform the same operation in all lanes

Neon SIMD instruction set extension performing up to 16 operations per instruction (optional).



■ What is different between VFP and NEON ?

- Neon is a SIMD (Single Instruction Multiple Data) accelerator processor as part of the ARM core. It means that during the execution of one instruction the same operation will occur on up to 16 data sets in parallel. Since there is parallelism inside the Neon, you can get more MIPS or FLOPS out of Neon than you can a standard SISD processor running at the same clock rate.
- The biggest benefit of Neon is if you want to execute operation with vectors, i.e. video encoding/decoding. Also it can perform single precision floating point(float) operations in parallel.
- VFP is a classic floating point hardware accelerator. It is not a parallel architecture like Neon. Basically, it performs one operation on one set of inputs and returns one output. It's purpose is to speed up floating point calculations. It supports single and double precision floating point.

High performance VFPv3 floating point unit doubling the performance of previous ARM FPUs

ARM Cortex A15 dual core with GPU : case study - Exynos 5250

Exynos 5 Dual (5250)

THE INDUSTRY'S FIRST CORTEX-A15 DUAL-CORE MOBILE PROCESSOR

The Exynos 5250 released in 2011 is the industry's first Cortex-A15 dual-core mobile processor built on 32nm HKMG process with WQXGA display support.

SAMSUNG



CPU

1.7GHz Dual-Core
(Cortex®-A15)



GPU

Mali™-T604 MP4



Process

32nm HKMG Process



Display

Up to WQXGA
(2560x1600)



Storage

eMMC 4.5



Memory

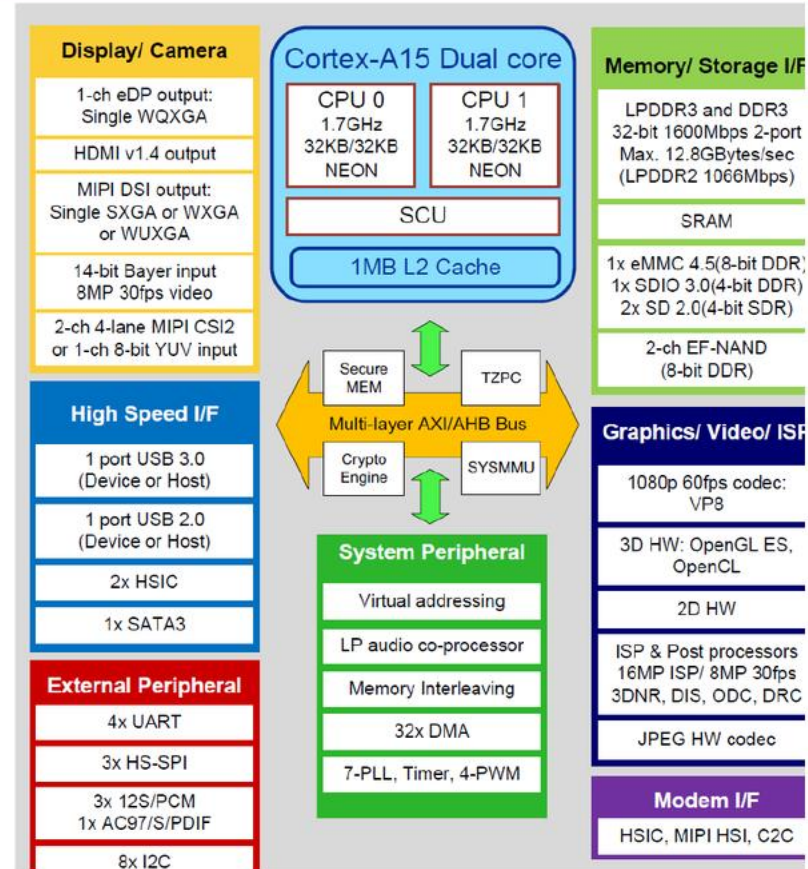


Camera

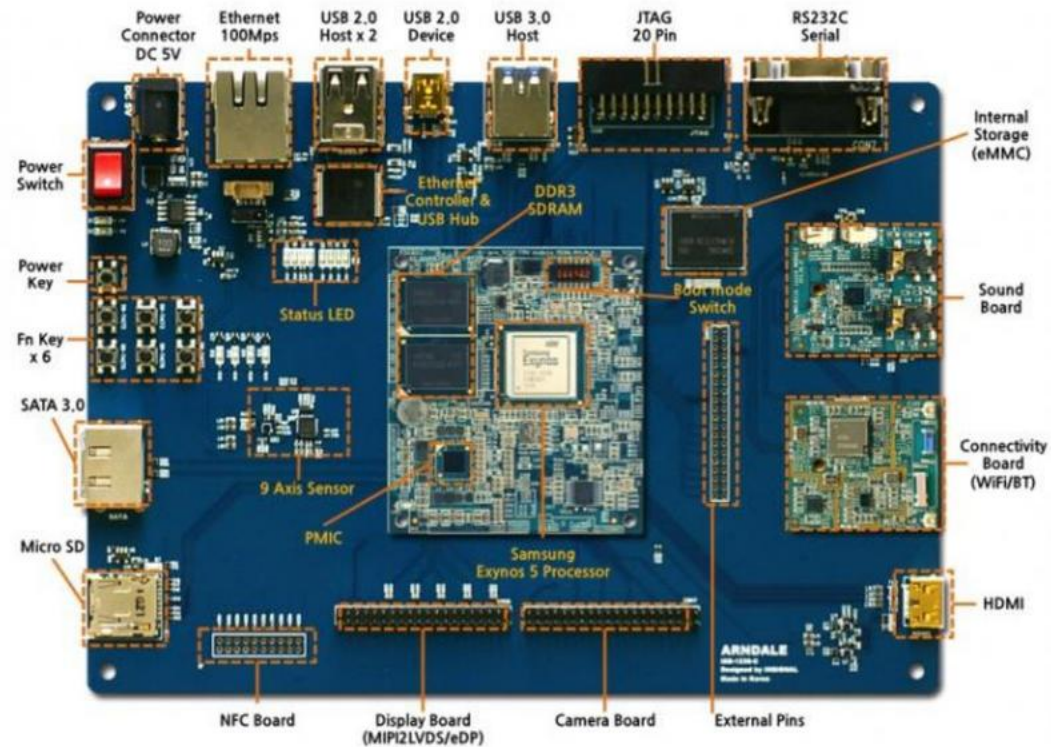


Video

- Samsung Exynos 5250 Dual
- CPU: ARM Cortex-A15 (2-core)
1.7 GHz 32nm
- 32KB L1 cache, 1MB L2 cache
- GPU: ARM MALI T604
- 64 concurrent threads
- Vector ALUs
- 128b registers
- OpenCL 1.1 Full Profile
- RAM: 2GB LP-DDR3 800 MHz
(12.8 GB/s)
- Truly unified cached memory
- CPU and GPU memory is shared – NO COPY!
- 128b wide L1 and L2 access
- 2 independent job queue in T628
(in Samsung Exynos 5 Octa)



■ Arndale board with Exynos 5250



■ Graphic Processing Unit

- A Graphics Processing Unit is a co-processor that takes on graphical calculations and transformations so that the main CPU does not have to be burdened by them.
- The use of a GPU can greatly increase the performance of a device, especially when used for tasks such as 3D gaming.
- A GPU can be a stand-alone chip or, as is more often the case, integrated into a complete chip design that includes one or more CPU cores.





Smooth DVD-Quality Video Playback



Sharp, Vivid Photos



Console-Class 3D Gaming

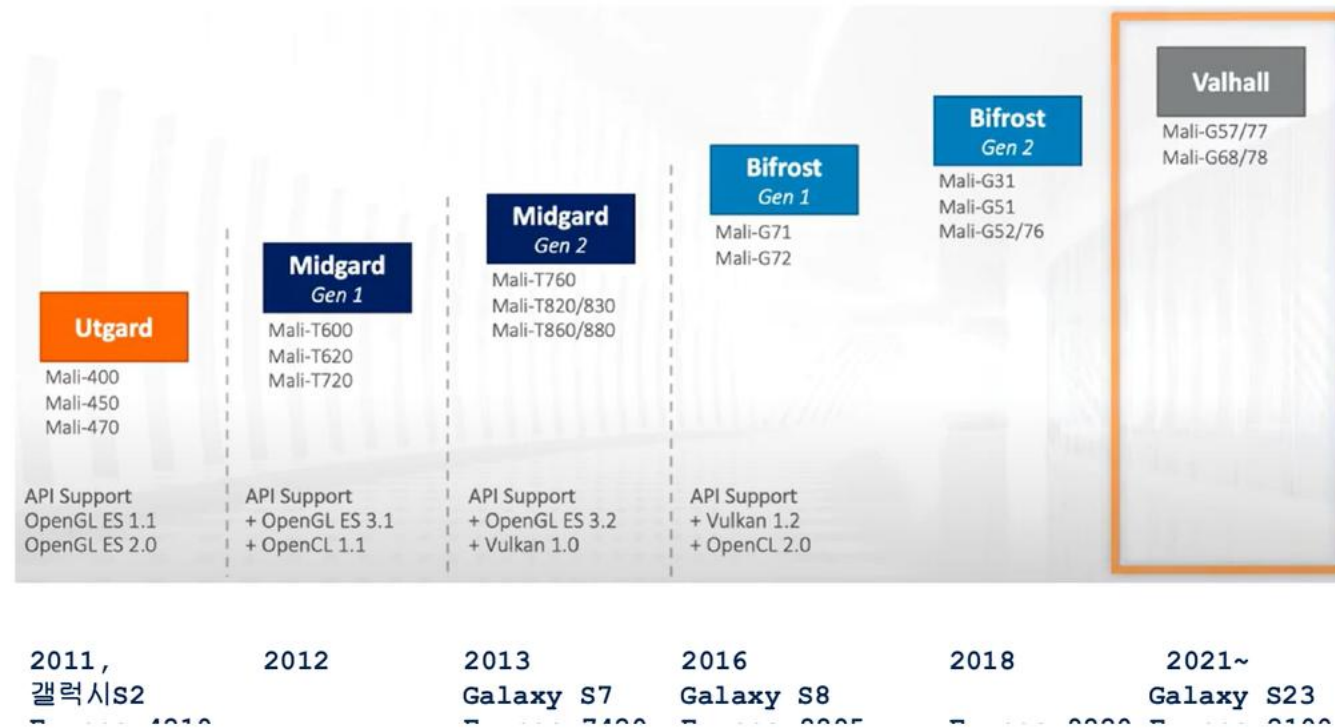


True, Fluid Digital TV



Less Battery Usage

■ The Mali Family



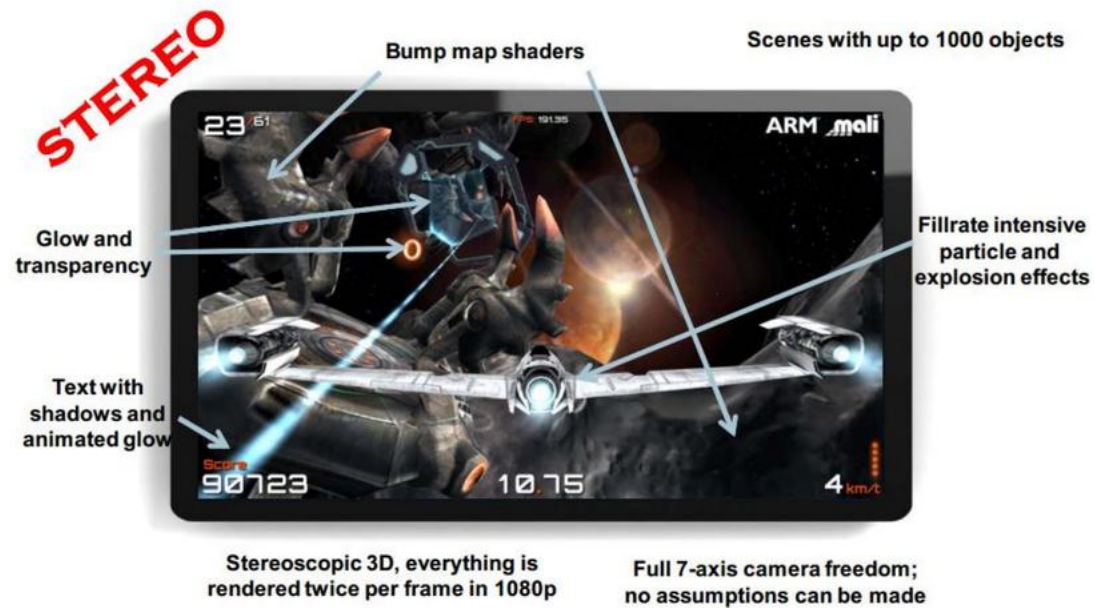
- **World's 1st embedded multi-core GPU with 4 cores**
(멀티코어 구성이 가능한 최초의 Mali GPU!!!!)
- **Configurable L2 cache tuned for maximum throughput of size 256 KB**
- **Multi-core scaling transparent to software developers**
- **Devices :Win Accord, SmartQ T10 , Samsung Galaxy S II**
- **Chipsets : Samsung Exynos 4210,, AML logic 8726-M, 8726-MX**

독립 셰이더 구조로, 버텍스를 담당하는 코어와 픽셀을 담당하는 코어가 각각 나누어져 있음.
버텍스 코어는 1개(코어 수에 포함되지 않고),
독립적으로 픽셀을 처리하는 Fragment Processor가 각각
추가됨 독립 셰이더 구조 때문에 떨어지는 버텍스 성능으로 인한 병목 현상이 일어나며

모바일에서는 버텍스 성능이 중요한 포리곤보다 픽셀 성능이 중요한 텍스처 연산이 더 많기



■ Mali-400 in 1080p at 60fps



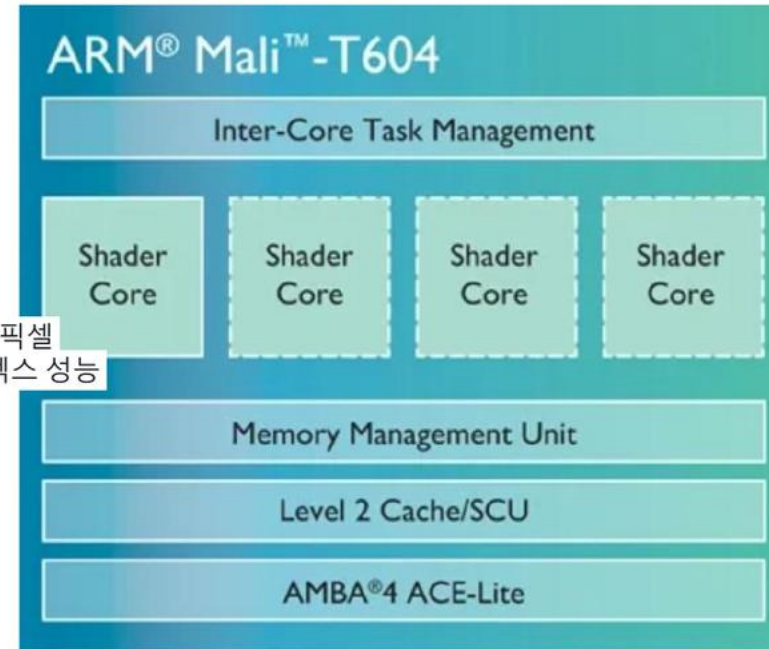
■ Mali T604

- Mali t604 is the 1세대 Midgard 아키텍처 기반 첫 GPU for ARM.
- It is 4 times better than other previous mali graphic processors & it can go to single core to quad core.

기존 아키텍처는 1개 코어만 버텍스 작업 담당-> 픽셀 코어와 동등한 버텍스 코어를 가짐, 4배 이상 버텍스 성능 향상

- It has been used in famous series of Google tablet nexus 10.

- Andreno 225, nvidia Tegra APX 2500, PowerVR SGX 530, Etc ... !!



통합 셰이더 구조로 인하여 각각의 코어에 코어 당 2개 연산유닛 텍스처 유닛이 별도로 들어

■ Mali-T604

25,000 Triangles Game Character

Sophisticated 100+ flops per pixel skin shader with sub-surface scattering approximation

2048x2048 texture resolution for incredible close-up detail everywhere

1920x1080p 4X FSAA in 60 fps on Mali-400 MP4 cores!



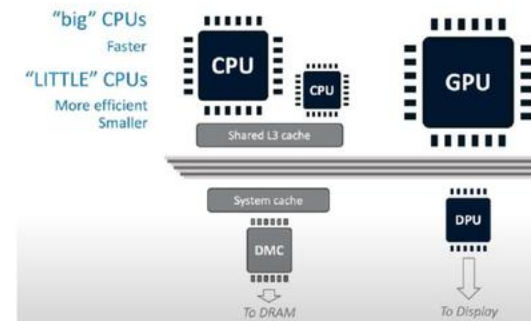
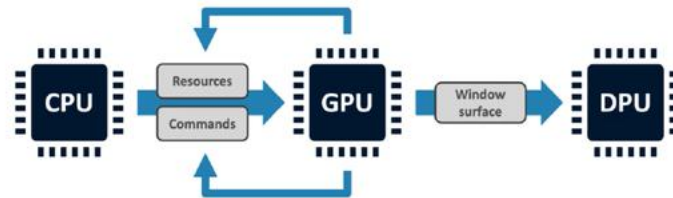
Skeletal animation calculated entirely on the Mali geometry processor

Per-pixel normal mapped lighting for extra detail

Material variation through custom shaders for different objects

<https://youtu.be/UL2yXqH4dEg?si=F0dlM5nr5RT8Icsy>

■ Arm Mali GPU Training Series Ep 1.1 ■ Introduction to mobile systems



https://youtu.be/tnR4mExVC1Y?si=I_THQekbMJVtjyjo

- Cortex-A9 MPCore

■ Variable length, out of order, superscalar pipeline

- Two instructions are fetched in one cycle
- Issue up to 4 instructions per cycle into:
 - Primary data processing pipeline
 - Secondary data processing pipeline
 - Load-store pipeline
 - Compute engine (FPU/NEON) pipeline

■ Speculative execution

- Supporting virtual renaming of physical registers and removing pipelines stalls due to data dependencies

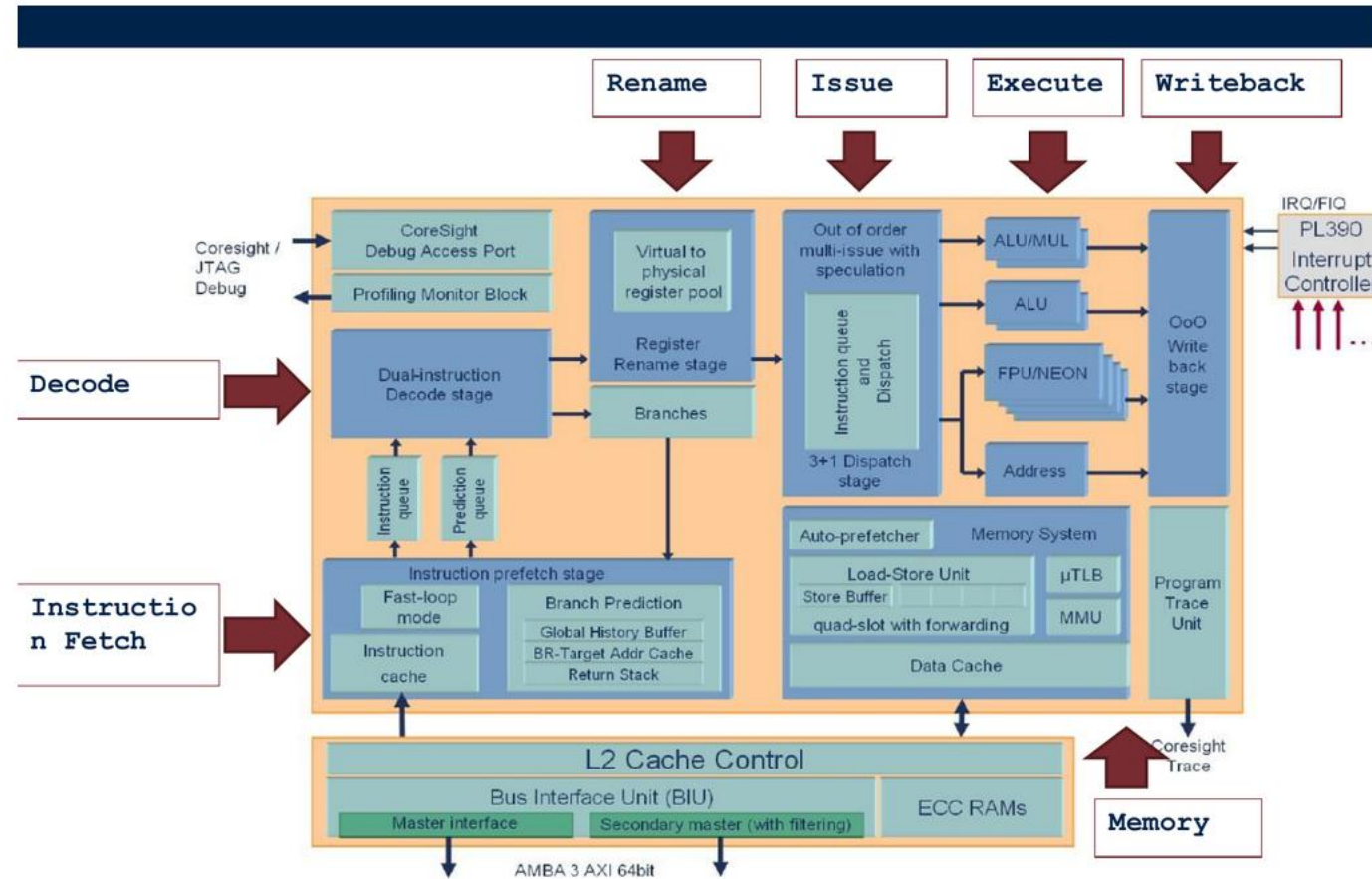
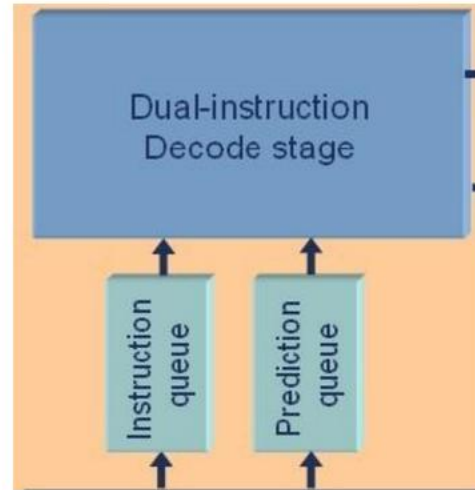


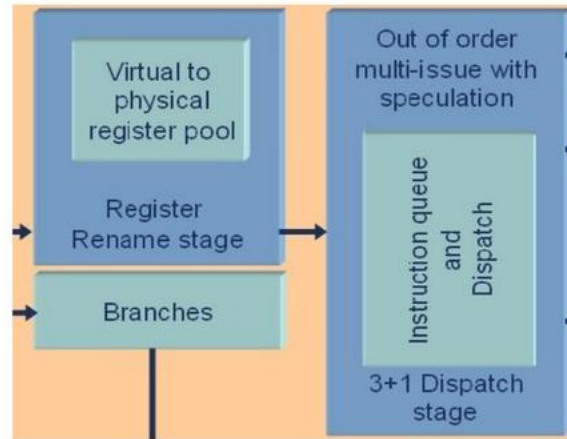
Fig. 1 Cortex-A9 microarchitecture structure and the single core interfaces.



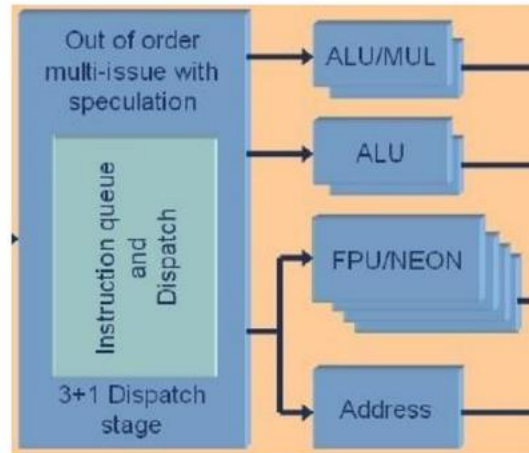
- **Instruction cache size:** 16KB, 32KB, or 64KB
- **Superscalar pipeline:** fetching two instructions at once
- **Branch Prediction:**
 - Global History Buffer: 1K ~ 16K entries
 - Branch-Target Address Cache: 512 ~ 4K entries
 - Return stack of 4 x 32 bits
- **Fast-loop mode:** instruction loop that are smaller than 64 bytes often complete without additional instruction cache accesses



- **Super Scalar Decoder**
 - Capable of decoding two full instructions per cycle



- **Register Renaming**
 - Resolving data dependencies and unroll small loops by hardware
 - Enabled by renaming of physical registers into an available pool of virtual registers.



- Issue can be fed maximum of 2 instructions per cycle
- Issue can dispatch up to 4 instructions per cycle
- Out of order selection of instructions from queue



- **Variable length Executing Stage (1 ~ 3 cycles)**
 - Most Instructions finish within 1 cycle
 - Instruction which folds shifts and rotates can take 3 cycles
 - ADD r0, r1, r2 (1 cycle)
 - ADD r0, r1, r2 LSL #2 (2 cycle)
 - Corresponds to $a = b + (c \ll 2)$;
 - ADD r0, r1, r2 LSL r3 (3 cycle)
 - Corresponds to $a = b + (c \ll d)$;

• NEON Media Processing Engine

- NEON technology supports instructions targeted primarily at audio, video, 3D graphics, image and speech processing.



Watch any video in
any format



Edit & Enhance
captured videos
Video stabilization



Antialiased rendering
& compositing



Advanced
User Interfaces



Game processing



Process
megapixel
photos quickly



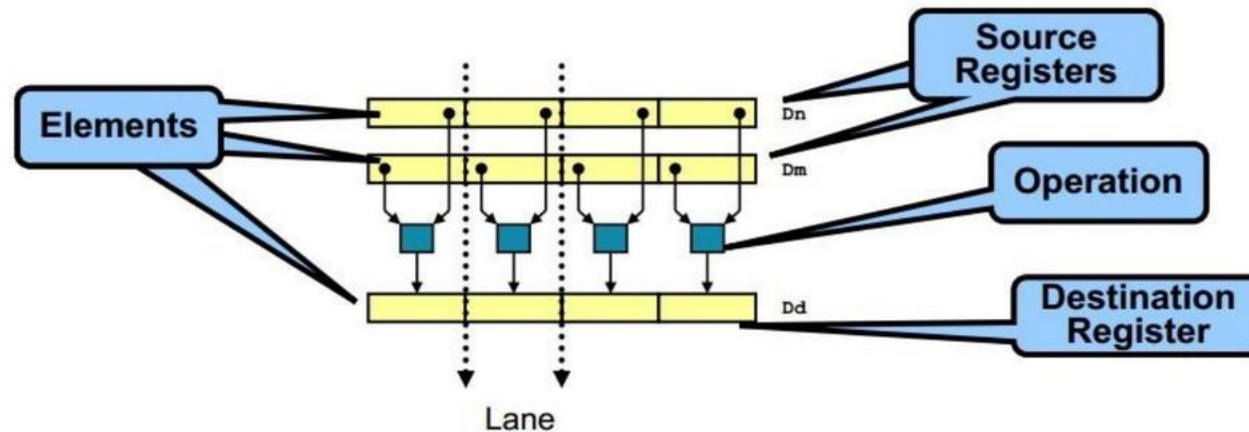
Voice recognition



Powerful multi-
channel hi-fi
audio processing

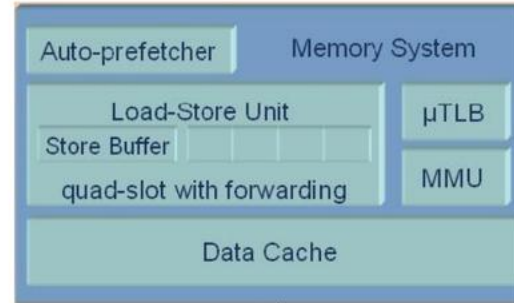
■ What is NEON?

- NEON is a wide SIMD data processing architecture
 - 32 registers, 64 bit wide or 16 registers, 128 bit wide
- NEON instructions perform “Packed SIMD” processing
 - Registers can be considered as “vector” of same data type
 - Instructions perform the same operation in all lanes

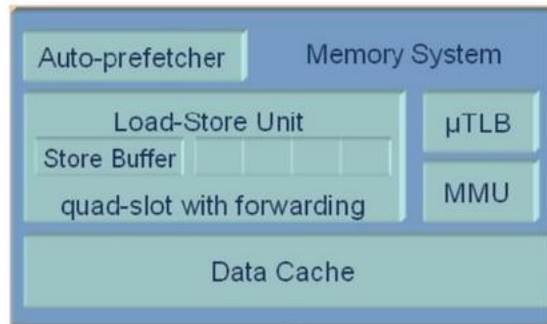


- **NEON Media Processing Engine supports vector computations on:**
 - half-precision (16bit), single-precision (32bit), double-precision (64bit) floating-point numbers
 - 8, 16, 32 and 64 bit signed and unsigned integers

- **Supported Operations Include:**
 - addition, subtraction, multiplication
 - maximum or minimum of a vector of operands
 - Inverse square-root approximation ($y = x^{-(1/2)}$)
 - many more

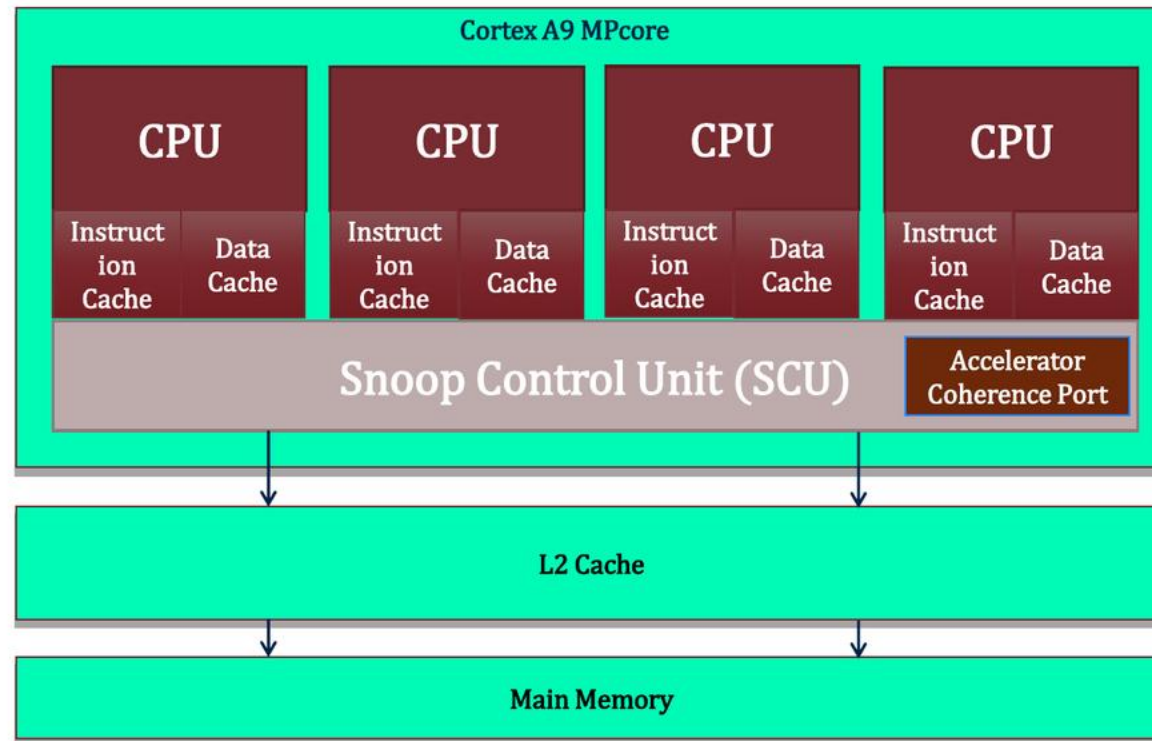


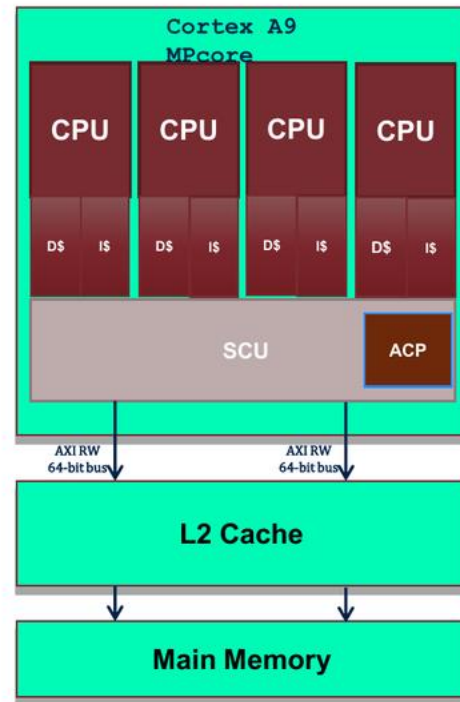
- dependent load-store instructions forwarded for resolution within memory system
- 2-level TLB structure
 - micro TLB
 - 32 entries on data side and 32 or 64 entries on instruction side
 - to reduce power consumed in translation and protection look-ups
 - main TLB



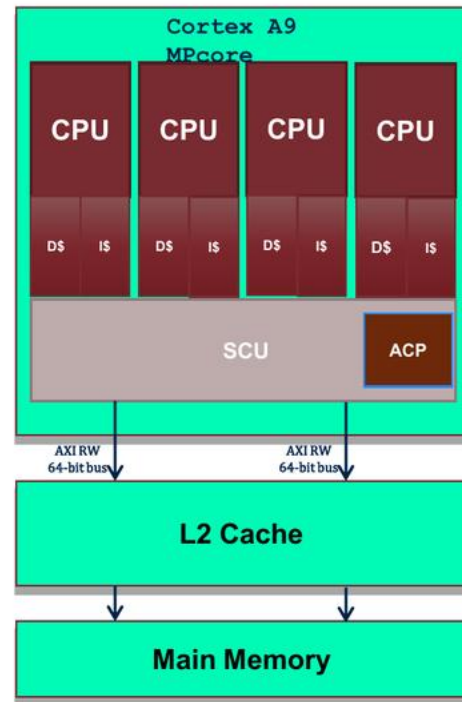
■ Data prefetcher

- monitor cache line requests by processor and cache misses to determine how much data to prefetch
- can prefetch up to 8 independent data streams
- prefetch and allocate data in the L1 data cache, as long as it keeps hitting in the prefetched cache line

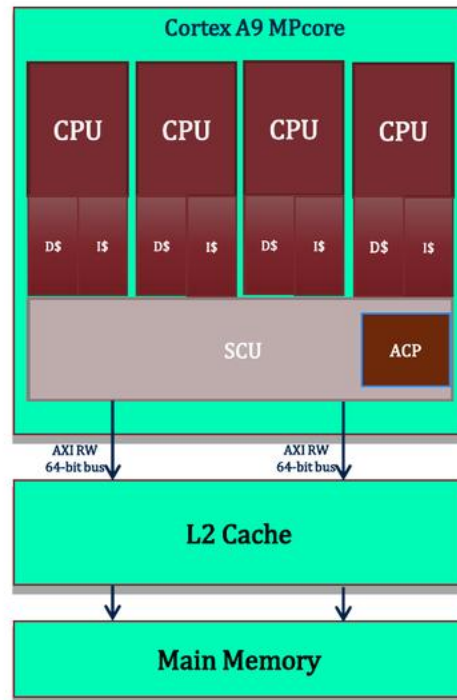




- **Non-unified**
 - 32 bytes line length
 - can be disabled independently
- **16, 32 or 64KB**
- **4 - way associative**
- **support for Security Extensions**
- **I cache: VIPT**
- **D cache: PIPT**
 - reduce number of caches flushes and refill and save energy



- shared, unified
- Off-chip
- 128KB to 8MB
- 4 to 16-way associative



■ Integral part of cache systems

■ Connects processors to memory system through AXI interfaces



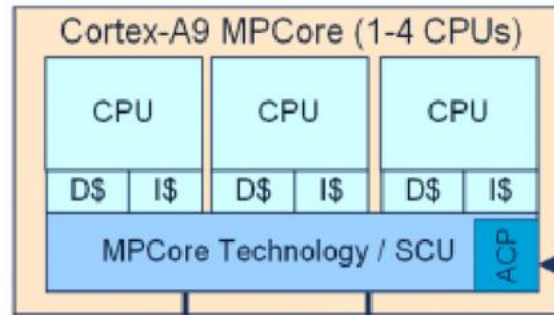
snoopy +

1. 형용사 [구어] 기웃거리며 돌아다니는; 이것저것 캐묻는, 참견하기 좋아하는
2. 명사 [S~] 스누피 ((C. Schulz의 만화 Peanuts에 나오는 개))

snoop (형용사: snoopy) +

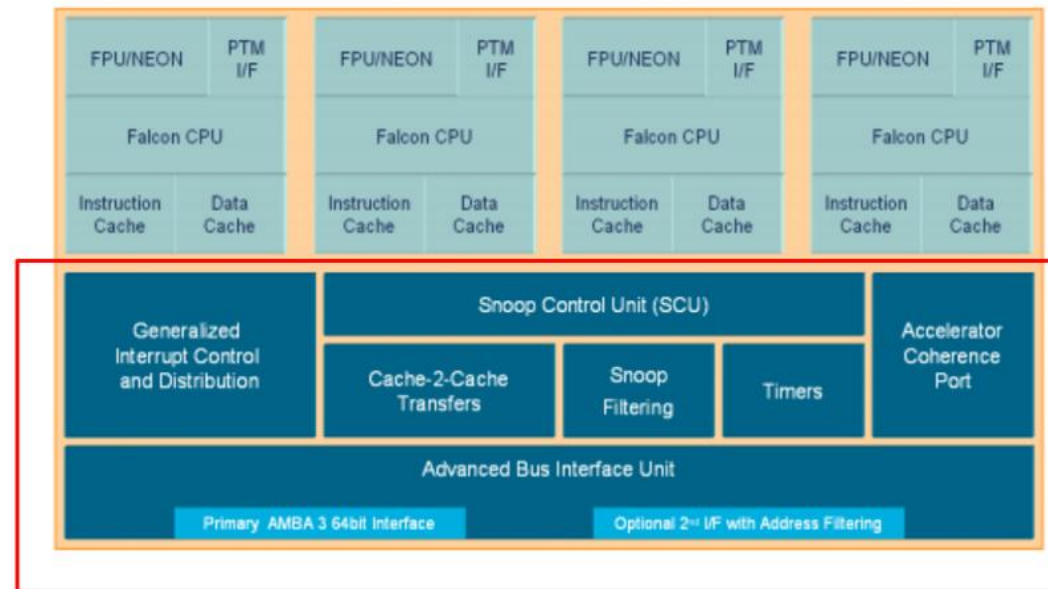
1. 동사 비격식 못마땅할 기웃거리다, 엿따하다
2. 명사 비격식 못마땅할 엿따꾼

동아출판 프리임 영한사전



- SCU functions :
 - maintain data cache coherency (does **not** support instruction cache coherency)
 - initiate L2 memory accesses
 - arbitrate between processors' simultaneous request for L2 accesses
 - manages accesses from ACP

Architecture



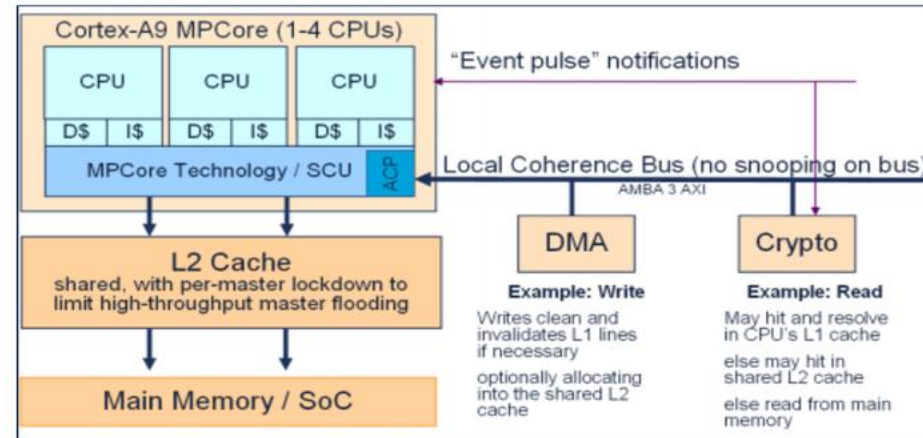
Cache to Cache Transfer : Core 1의 data cache miss 인 cache line을 Core2의 data cache 에서 갖고 있을 때 L2나 Main Memory 까지 가지 않고 Core2의 Cache line을 Core1의 Cache line 으로 복사

<http://www.arm.com/files/pdf/ARMCortexA-9Processors.pdf>

■ Cache Coherence

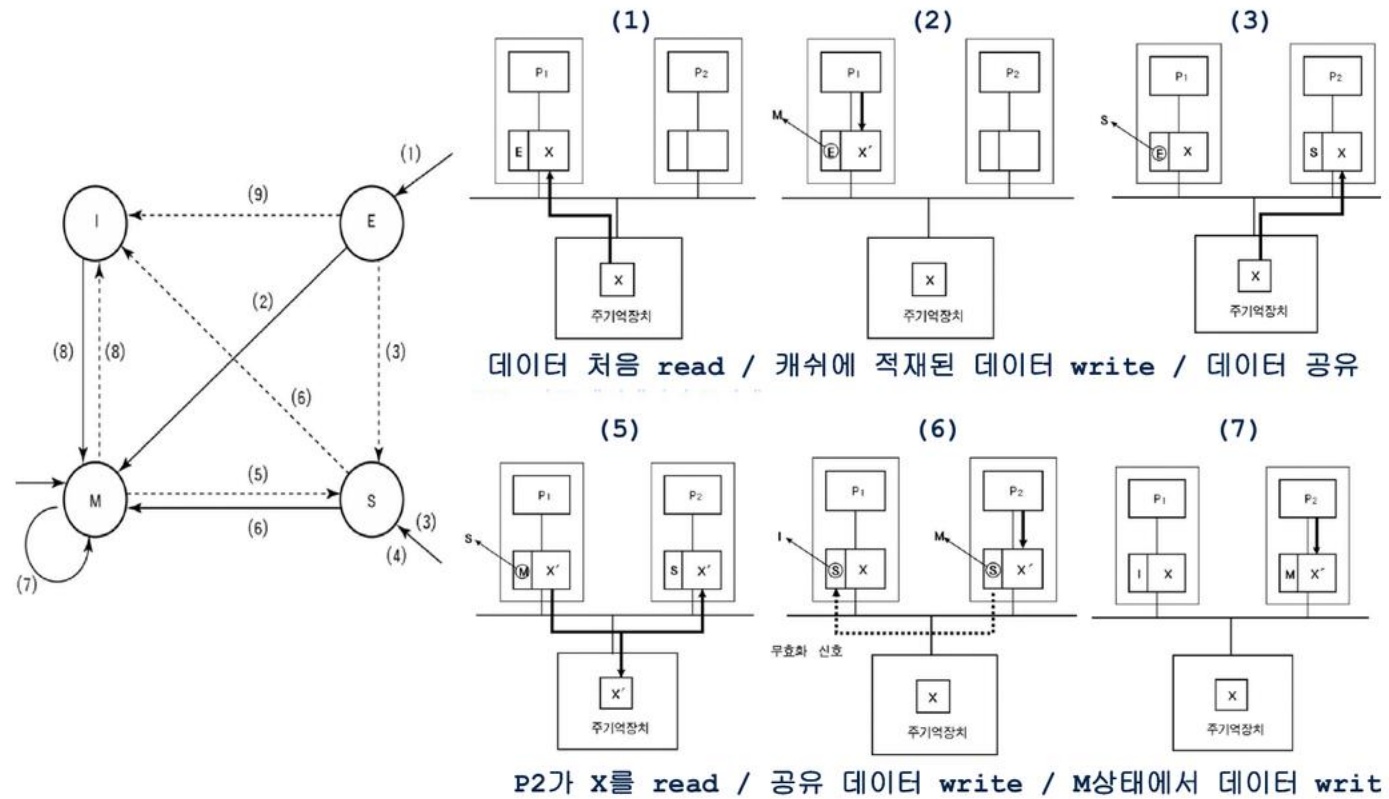
- Shared memory multi-core 구조에서는 복수의 Processor core가 Main Memory를 공유함
- Cache를 사용하는 SMP System에서는 메모리의 내용 변경 시 각 Core가 가지고 있는 Local cache에서 먼저 변경하고, Cache strategy에 따라 Write back 또는 Write Through 가능
- Main Memory에 Write하는 것은 시간이 소요되는 작업인데, 코어 P1에서 A라는 주소에 X라는 값을 Write했을 때 Write가 Complete되지 않은 상태이거나, Write back strategy에 의해서 A에 X를 write하지 않은 시점에 코어 P2가 동일 메모리 주소의 값을 Read하면 inconsistency 발생
- 이와 같이, global state(main memory)와 local state(core 별 cache)의 2가지 state의 불일치 문제를 해결하기 위한 mechanism

Coherence Port

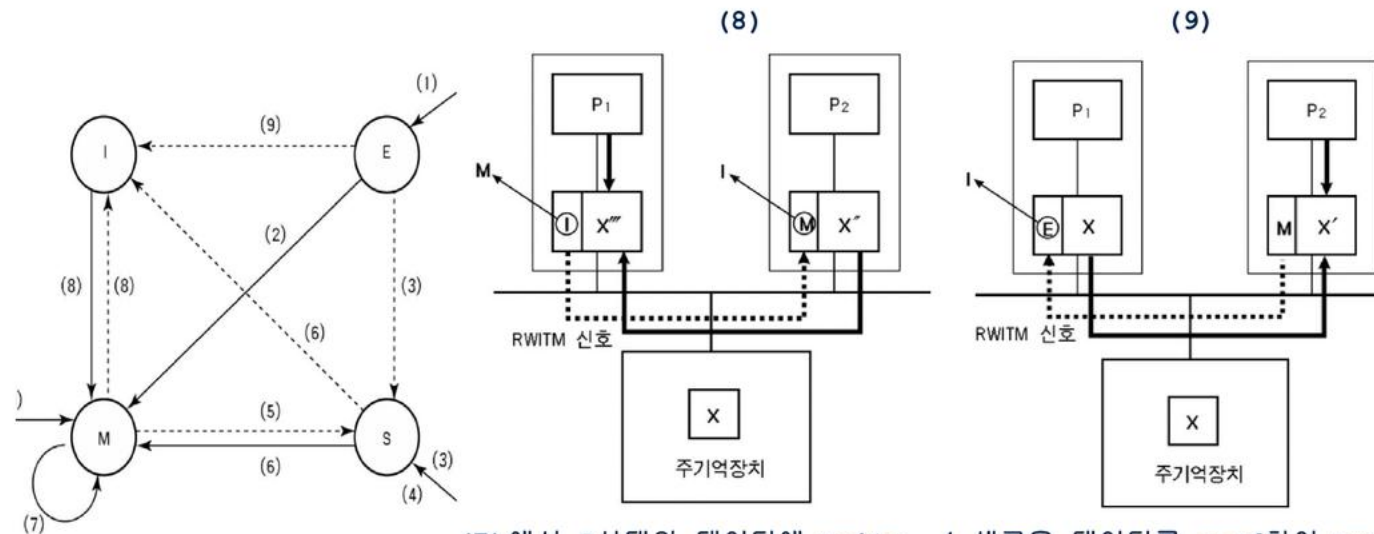


- optional AXI 64-bit slave port
- allows to connect to non-cached system mastering peripherals and accelerators
 - DMA engine or cryptographic accelerator
- SCU enforces memory coherency

MESI



MESI



(7)에서 I상태의 데이터에 write / 새로운 데이터를 read하여 wri

<Write miss 발생 case>

Control

- Which core services interrupts?
- GIC gives the programmer control
- Centralizes interrupts, then dispatches to individual core(s)

System Level Benchmarking Analysis of the Cortex A9 MPCore
Roberto Mijat, ARM Connected Community Technical Symposium, 2009

■ Execution depending on condition status flags

- 코어가 특정 명령어를 실행할 여부를 제어 가능
- 대부분의 명령어는 코어가 그것을 실행할 지 여부를 결정 수 있는 조건 인자를 가짐
- 명령어 실행 전 코어는 자신의 조건인자와 `cpsr` 상태 플래그를 비교

■ Condition mnemonics

- Ep 572, Appendix. Table A.2, ARM System Developers' guide

■ How do we create conditional statements in ARM assembly?

■ E.g.,

```
If ( i < 10 ) {
    c=i+'0';
} else {
    c=i+'A'-10;
```

```
CMP i, #10
ADDLO c, i, #'0'
ADDHS c, i, #'A'-10
```

Table A.2 ARM condition mnemonics.

<cond>	Instruction is executed when	<i>cpsr</i> condition
{ AL}	ALways	TRUE
EQ	EQual (last result zero)	Z==1
NE	Not Equal (last result nonzero)	Z==0
{CS HS}	Carry Set, unsigned Higher or Same (following a compare)	C==1
{CC LO}	Carry Clear, unsigned LOwer (following a comparison)	C==0
MI	MInus (last result negative)	N==1
PL	PLus (last result greater than or equal to zero)	N==0
VS	V flag Set (signed overflow on last result)	V==1
VC	V flag Clear (no signed overflow on last result)	V==0
HI	unsigned Higher (following a comparison)	C==1 && Z==0
LS	unsigned Lower or Same (following a comparison)	C==0 Z==1
GE	signed Greater than or Equal	N==V
LT	signed Less Than	N!=V
GT	signed Greater Than	N==V && Z==0
LE	signed Less than or Equal	N!=V Z==1
NV	NeVer—ARMv1 and ARMv2 only—DO NOT USE	FALSE

- <NoUnsignedOverflow> is the same as 1 |— <UnsignedOverflow>.
- <Zero> is a flag indicating that the result of an arithmetic or logical operation is zero. The Z flag in the *cpsr* typically records the zero condition.
- <Negative> is a flag indicating that the result of an arithmetic or logical operation is negative. In other words, <Negative> is bit 31 of the result. The N flag in the *cpsr* typically records this condition.

- ARM instructions can be made to execute conditionally by postfixing them with the appropriate condition code field.

- This improves code density *and* performance by reducing the number of forward branch instructions.

```

CMP    r3,#0
BEQ    skip
ADD    r0,r1,r2
skip

```

```

CMP    r3,#0
ADDNE  r0,r1,r2

```

- By default, data processing instructions do not affect the condition code flags but the flags can be optionally set by using “S”. CMP does not need “S”.

```

loop
...
SUBS   r1,r1,#1
BNE    loop

```

decrement r1 and set flags

if Z flag clear then branch

일반적인 data processing instruction은 condition code flag에 영향을 주지 않으나, 'S' suffix를 선택적으로 설정함으로써 가능함. (CMP 명령은 'S' suffix 불필요)

■ The possible condition codes are listed below:

- Note AL is the default and does not need to be specified

Suffix	Description	Flags tested
EQ	Equal	Z=1
NE	Not equal	Z=0
CS/HS	Unsigned higher or same	C=1
CC/LO	Unsigned lower	C=0
MI	Minus	N=1
PL	Positive or Zero	N=0
VS	Overflow	V=1
VC	No overflow	V=0
HI	Unsigned higher	C=1 & Z=0
LS	Unsigned lower or same	C=0 or Z=1
GE	Greater or equal	N=V
LT	Less than	N!=V
GT	Greater than	Z=0 & N=V
LE	Less than or equal	Z=1 or N!=V
AL	Always	

■ Use a sequence of several conditional instructions

```
if (a==0) func(1);  
    CMP     r0,#0  
    MOVEQ   r0,#1  
    BLEQ    func
```

■ Set the flags, then use various condition codes

```
if (a==0) x=0;  
if (a>0)  x=1;  
    CMP     r0,#0  
    MOVEQ   r1,#0  
    MOVGT   r1,#1
```

■ Use conditional compare instructions

```
if (a==4 || a==10) x=0;  
    CMP     r0,#4  
    CMPNE   r0,#10  
    MOVEQ   r1,#0
```

- Branch : `B{<cond>} label`
- Branch with Link : `BL{<cond>} subroutine_label`



- The processor core shifts the offset field left by 2 positions, sign-extends it and adds it to the PC
 - ± 32 Mbyte range
 - How to perform longer branches?

■ Consist of :

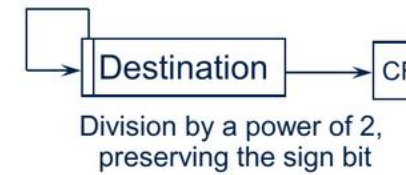
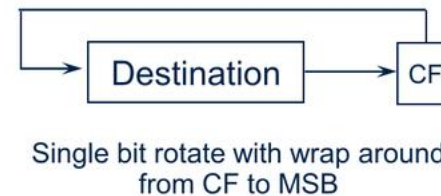
- | | | | | | | |
|------------------|-----|-----|-----|-----|-----|-----|
| ■ Arithmetic: | ADD | ADC | SUB | SBC | RSB | RSC |
| ■ Logical: | AND | ORR | EOR | BIC | | |
| ■ Comparisons: | CMP | CMN | TST | TEQ | | |
| ■ Data movement: | MOV | MVN | | | | |

- These instructions only work on registers, NOT memory.

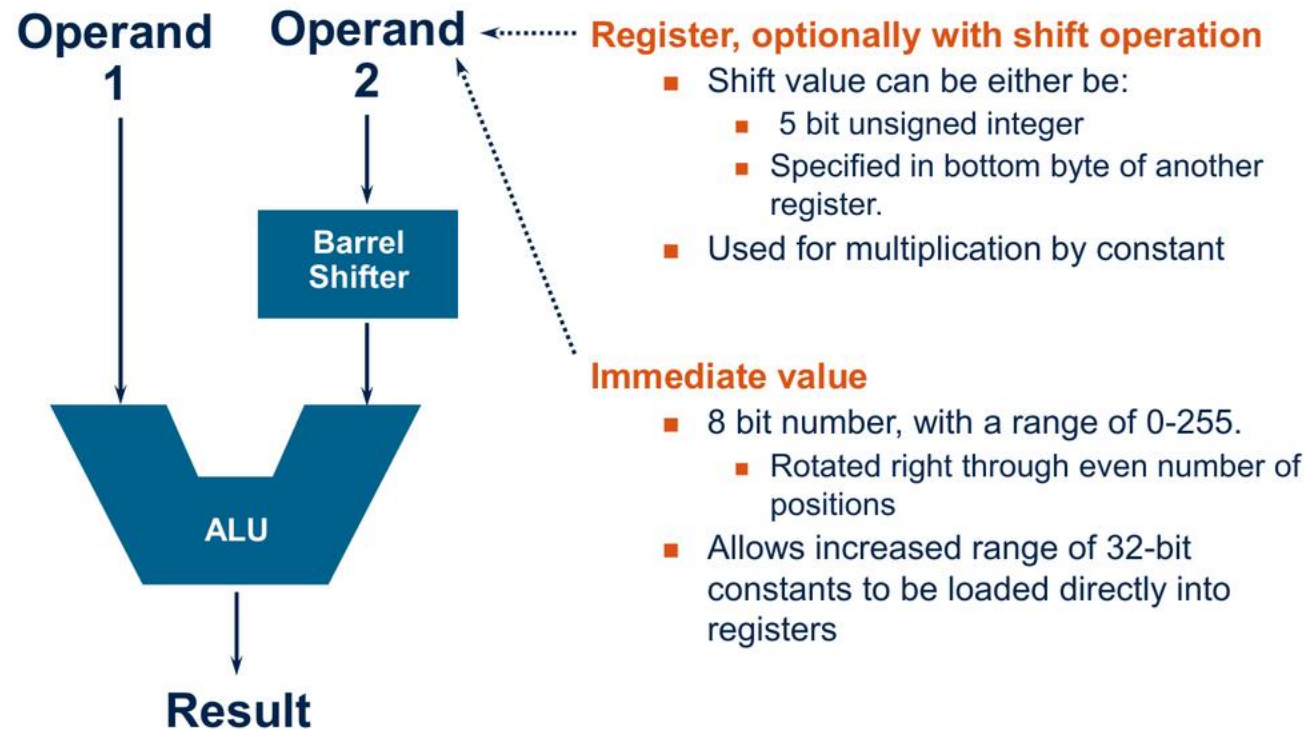
■ Syntax:

`<Operation>{<cond>}{S} Rd, Rn, Operand2`

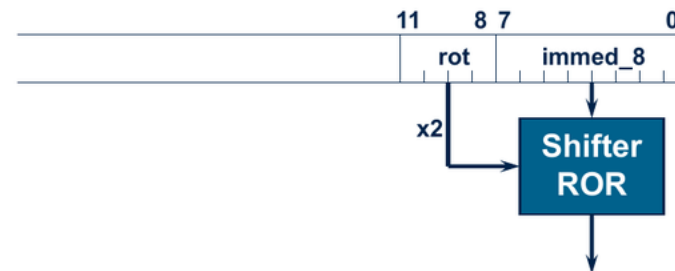
- Comparisons set flags only - they do not specify Rd
 - Data movement does not specify Rn
- Second operand is sent to the ALU via barrel shifter.

LSL : Logical Left Shift**ASR: Arithmetic Right Shift****LSR : Logical Shift Right****ROR: Rotate Right****RRX: Rotate Right Extended**

The Second Operand



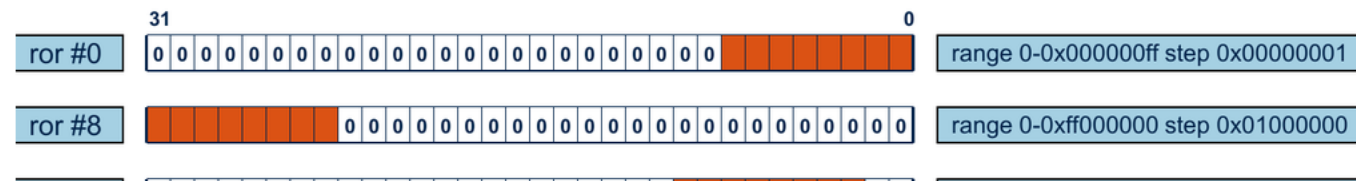
- No ARM instruction can contain a 32 bit immediate constant
 - All ARM instructions are fixed as 32 bits long
- The data processing instruction format has 12 bits available for operand2



다음에 사용된 상수의 validity는?

```
mov r0, #0x7f00
mov r0, #0x30c0
mov r0, #0x3f000000
mov r0, #12345678
mov r3, #0x14e
```

- 32비트 명령내에 상수 값을 함께 저장 -> 범위 제한 발생
- 8비트 값과 짝수비트 ROR(Rotate Right)로 표현되는 값
- 0~255 범위의 상수는 무조건 가능



- To allow larger constants to be loaded, the assembler offers a pseudo-instruction:

- `LDR rd, =const`

- This will either:

- Produce a **MOV** or **MVN** instruction to generate the value (if possible).

or

- Generate a **LDR** instruction with a PC-relative address to read the constant from a *literal pool* (Constant data area embedded in the code).

- For example

■ <code>LDR r0,=0xFF</code>	=>	<code>MOV r0,#0xFF</code>
■ <code>LDR r0,=0x55555555</code>	=>	<code>LDR r0,[PC,#Imm12]</code>
		...
		...
		<code>DCD 0x55555555</code>

범위 제한 없는 32비트 값으로 표현 가능
실제 상수값은 메모리에 데이터를 위한 메모리 할당 **directive** (define constant double word : DCD) 활용하여 레지스터 참조로 변환됨

LDR	STR	Word
LDRB	STRB	Byte
LDRH	STRH	Halfword
LDRSB		Signed byte load
LDRSH		Signed halfword load

- **Memory system must support all access sizes**

- **Syntax:**

- **LDR**{<cond>}{<size>} Rd, <address>
- **STR**{<cond>}{<size>} Rd, <address>

e.g. **LDREQB**

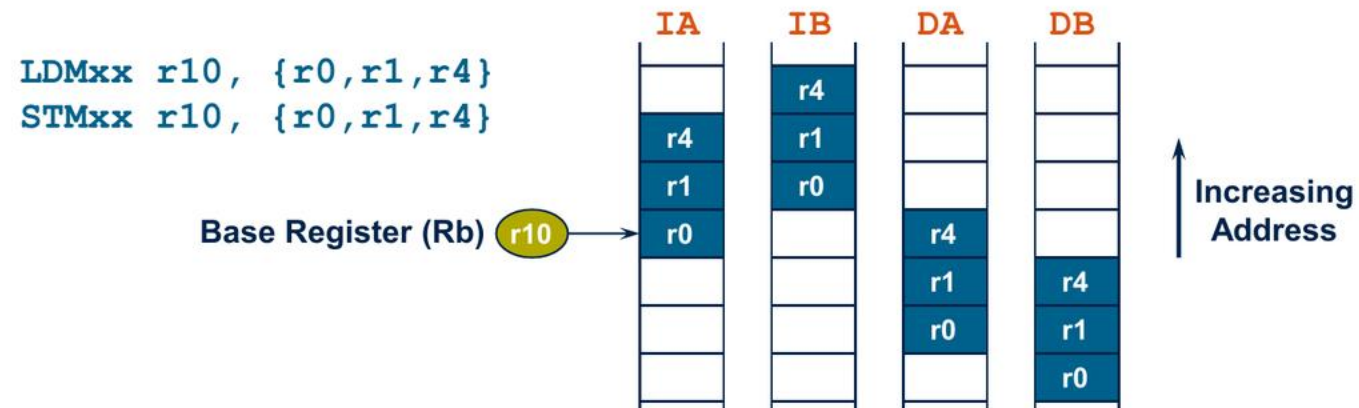
- **Address accessed by LDR/STR is specified by a base register plus an offset**
- **For word and unsigned byte accesses, offset can be**
 - An unsigned 12-bit immediate value (ie 0 - 4095 bytes).
`LDR r0, [r1, #8]`
 - A register, optionally shifted by an immediate value
`LDR r0, [r1, r2]`
`LDR r0, [r1, r2, LSL#2]`
- **This can be either added or subtracted from the base register:**
`LDR r0, [r1, #-8]`
`LDR r0, [r1, -r2]`
`LDR r0, [r1, -r2, LSL#2]`
- **For halfword and signed halfword / byte, offset can be:**
 - An unsigned 8 bit immediate value (ie 0-255 bytes).
 - A register (unshifted).
- **Choice of *pre-indexed* or *post-indexed* addressing**

■ Syntax:

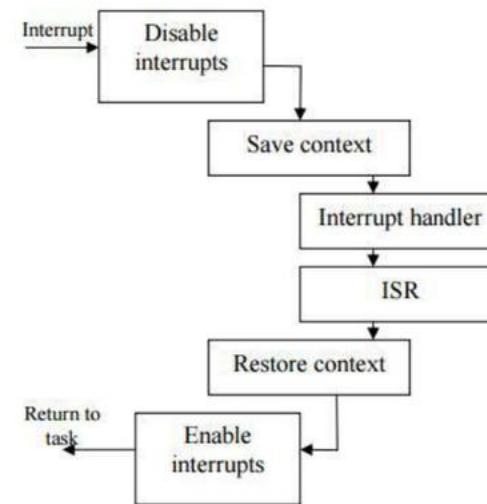
<LDM | STM>{<cond>}<addressing_mode> Rb{!}, <register list>

■ 4 addressing modes:

LDMIA / STMIA	increment after
LDMIB / STMIB	increment before
LDMDA / STMDA	decrement after
LDMDB / STMDB	decrement before



- Exceptions and interrupt control
- Exceptions are events that cause changes to program flow .
- When one happens, the processor suspends the current executing task and executes a part of the program called the exception handler.
- After the execution of the exception handler is completed, the processor then resumes normal program execution.



P334, ARM System Developer's Guide

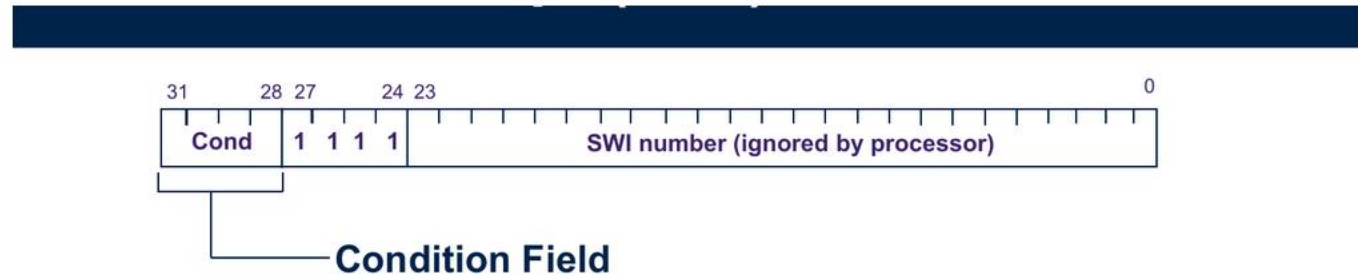
The simplest interrupt handler is a handler that is nonnested: the interrupts are disabled until control is returned back to the interrupted task or process. Because a nonnested interrupt handler can only service a single interrupt at a time

- A Software interrupt instructions (SWI) causes a software interrupt exception, which provides a mechanism for applications to call operating system routines.
- Each SWI instruction has an associated SWI number, which is used to represent a particular function call or feature.

Syntax: `SWI{<cond>} SWI_number`

SWI	software interrupt	$lr_svc = \text{address of instruction following the SWI}$ $spsr_svc = cpsr$ $pc = \text{vectors} + 0x8$ $cpsr \text{ mode} = SVC$ $cpsr I = 1$ (mask IRQ interrupts)
-----	--------------------	---

P73, ARM System Developer's Guide



- Causes an exception trap to the SWI hardware vector
- The SWI handler can examine the SWI number to decide what operation has been requested.
- By using the SWI mechanism, an operating system can implement a set of privileged operations which applications running in user mode can request.
- Syntax:
 - `SWI{<cond>} <SWI number>`

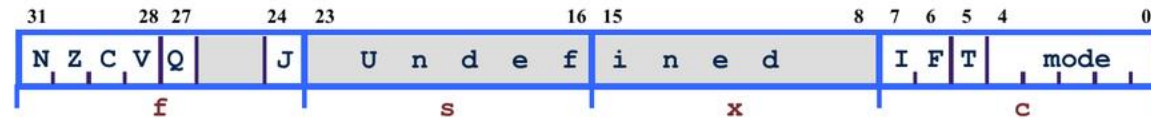
- We have a simple example of an SWI call with SWI number 0x123456, used by ARM toolkits as a debugging SWI. Typically the SWI instruction is executed in user mode.

0x00008000 SWI 0x123456

PRE cpsr = nzcVqift_USER
pc = 0x00008000
lr = 0x003fffff; lr = r14
r0 = 0x12

POST cpsr = nzcVqlft_SVC
spsr = nzcVqift_USER
pc = 0x00000008
lr = 0x00008004
r0 = 0x12

- SWI instructions are used to call operating system routines, you need some form of parameter passing. This is achieved using registers. In this example, register r0 is used to pass the parameter 0x12. The return values are also passed back via registers.
- Code called the SWI handler is required to process the SWI call. The handler obtains the SWI number using the address of the executed instruction, which is calculated from the link register lr.



- MRS and MSR allow contents of CPSR / SPSR to be transferred to / from a general purpose register.
- Syntax:
 - **MRS**{<cond>} Rd,<psr> ; Rd = <psr>
 - **MSR**{<cond>} <psr[_fields]>,<Rm> ; <psr[_fields]> = <Rm>
- where
 - <psr> = CPSR or SPSR
 - [_fields] = any combination of 'fsxc'
- Also an immediate form
 - **MSR**{<cond>} <psr_fields>,#Immediate
- In User Mode, all bits can be read but only the condition flags (_f) can be written.

- **B <label>**
 - PC relative. ± 32 Mbyte range.
- **BL <subroutine>**
 - Stores return address in LR
 - Returning implemented by restoring the PC from LR
 - For non-leaf functions, LR will have to be stacked

